EECE 2323 Digital Logic Design Lab

Lab Report 7: Adding Branch Logic to the Datapath to Complete Your Computer

Name: Arefin Mahmood

Date: 06/17/2026

Background & Purpose

In this lab, I completed the Lab 6 computer by adding branch support to the Program Counter. In Lab 6, the computer could fetch instructions from instruction memory and step through straight-line code, but the PC only moved to the next instruction. Lab 7 added the ability to change the next PC value using branch instructions. This matters because loops, if-statements, and real programs need control flow, not just sequential execution. The main goal was to update the PC so it uses PC + 1 for normal instructions and PC + signed offset when take_branch is high. I also loaded a signed multiplier program into instruction memory to test that the branch logic works in hardware.

Design Analysis

The main hardware change in this lab was the Program Counter. In Lab 6, the PC only moved to the next instruction each clock press. In Lab 7, I updated the PC so it supports branching. If take_branch is 0, the PC increments by 1. If take_branch is 1, the PC adds the signed branch offset and jumps to the target instruction. The PC was also changed to 9 bits because the branch offset is 9 bits. I also checked the decoder sign extension for addi, srai, beqz, and bnez so signed immediates and branch offsets work correctly.

My prelab program used repeated addition to multiply signed 8-bit numbers. The final hardware version had to match the lab instruction format, so I used instructions like add rd, rd, rs2 and addi rd, rd, nzimm. I also removed the unsupported j instruction and used a branch instruction instead. In the final program, x8 holds the multiplicand, x9 holds the multiplier, x10 holds the product, and x11 is the loop counter.

The final hardware test used 4 times 3. The program loaded x8 with 4 and x9 with 3, copied the multiplier into x11, then ran the positive loop. Each loop added x8 into x10 and subtracted 1 from x11. The product changed from 0 to 4, then 8, then 12, which is 000C in hex. The program also includes a negative multiplier path using srai, bnez, xor, addi, and a count-up loop, but that section was skipped in this test because the multiplier was positive.

Results and Analysis

The VIO results show that the updated instruction memory loaded the final program correctly and that the datapath followed the expected instruction sequence. At PC 000, the instruction initialized x8 to 4, which was used as the multiplicand. At PC 001, the instruction initialized x9 to 3, which was used as the multiplier. At PC 002, the multiplier was copied into x11, so x11 became the loop counter with a starting value of 3. This setup matched the intended test case of 4 times 3. The branch at PC 003 was not taken because x11 was not zero, so the program did not skip straight to done. At PC 004, the sign of the multiplier was checked using srai. Since the multiplier was positive, x9 became 0. Because of that, the bnez instruction at PC 005 was also not taken, and the program entered the positive multiplication loop.

The positive loop worked as expected. At PC 006, the program added x8 into x10. Since x8 was 4, the product register increased by 4 each time the loop ran. At PC 007, the counter x11 decreased by 1. At PC 008, the bnez instruction checked whether the counter was still nonzero. While x11 was

2 and then 1, take_branch was asserted and the PC returned to PC 006. This showed that the backward branch worked correctly. After three loop passes, x10 reached 000C, which is 12 in decimal. This matches the expected product for 4 × 3.

The final branch behavior was verified after the counter reached zero. When x11 became 0, the bnez instruction at PC 008 was not taken, so the PC moved forward to PC 009. At PC 009, the beqz instruction was true because x11 was 0, so take_branch became 1 and the PC jumped to PC 00F. This confirmed that the PC can update normally when no branch is taken and can also jump to a branch target when take_branch is active. Overall, the VIO screenshots show that instruction fetching, decoding, register updates, ALU operations, and branch-based PC updates all worked together for the final positive multiplication test.

Conclusion & Recommendations

This lab completed the Lab 6 computer by adding branch support to the Program Counter. The final design was able to fetch instructions from instruction memory, decode each instruction, update registers, perform ALU operations, and change the PC based on branch conditions. The VIO screenshots showed the program starting at PC 000, loading the correct instructions from the COE file, and stepping through the positive multiplication test for 4 times 3. The product register x10 reached 000C, which matched the expected decimal result of 12.

The most important part of this lab was confirming that the PC no longer only increments by 1. When a branch was not taken, the PC moved to the next instruction normally. When a branch was taken, the PC used the signed offset and jumped to the correct target address. This was shown during the positive loop, where bnez sent the PC back to PC 006 while the counter was not zero. It was also shown at PC 009, where beqz sent the PC to the done instruction after the counter reached zero. These results show that the processor now supports loops and conditional program flow.

Based on the results, I recommend testing this design one instruction at a time using BTN1 and saving VIO screenshots for the important PC values. The most useful signals to show are pc, instruction, opcode, rd, rs1, rs2, nzimm, offset, alu_take_branch, alu_output, and regfile_WriteData. These signals make it easier to explain what each instruction is doing and prove that the PC, instruction decoder, ALU, and register file are working together. I would also test both a positive multiplier and a negative multiplier in future testing, since the final program includes both paths.

Appendix A: Prelab

A.1 Handout Questions

Lab 7 completes the computer by adding branch logic to the Program Counter. The signed multiplier program uses repeated addition. Register x8 stores the multiplicand, x9 stores the multiplier and later the sign mask, x10 stores the product, and x11 stores the loop counter. If the multiplier is zero, the program goes to done. If the multiplier is positive, the product is created by adding x8 while x11 counts down. If the multiplier is negative, the program converts x8 to its two's complement negative value and then adds it while x11 counts up toward zero.

Register	Purpose
x8	Multiplicand
x9	Multiplier at first, then sign mask
x10	Product
x11	Counter

Table A1: Register use for the signed multiplication program.

A.2 Final Signed Multiplication Assembly Program

```
init:
   addi x8, x8, 4        # set x8 to 4
   addi x9, x9, 3        # set x9 to 3
   add  x11, x11, x9     # copy x9 into counter x11
   beqz x11, done        # if counter is 0, go to done
   srai x9, x9, 7        # check if multiplier is negative
   bnez x9, neg_setup    # if negative, go to negative setup

pos_loop:
   add  x10, x10, x8     # add x8 to product x10
   addi x11, x11, -1     # subtract 1 from counter
   bnez x11, pos_loop    # repeat if counter is not 0
   beqz x11, done        # go to done when counter is 0

neg_setup:
   xor  x8, x8, x9       # flip x8 bits
   addi x8, x8, 1        # make x8 negative

neg_loop:
   add  x10, x10, x8     # add negative x8 to product
   addi x11, x11, 1      # add 1 to counter
   bnez x11, neg_loop    # repeat if counter is not 0

done:
   addi x10, x10, 0      # keep final result in x10
```

PC	Instruction	Meaning
000	addi x8, x8, 4	Set multiplicand to 4
001	addi x9, x9, 3	Set multiplier to 3
002	add x11, x11, x9	Copy multiplier into counter
003	beqz x11, done	Go to done if counter is zero
004	srai x9, x9, 7	Check sign of multiplier
005	bnez x9, neg_setup	Go to negative setup if multiplier is negative
006	add x10, x10, x8	Add multiplicand to product
007	addi x11, x11, -1	Count down
008	bnez x11, pos_loop	Repeat positive loop if counter is not zero
009	beqz x11, done	Go to done after positive loop
00A	xor x8, x8, x9	Flip bits for negative multiplicand
00B	addi x8, x8, 1	Finish two's complement negative value
00C	add x10, x10, x8	Add negative multiplicand

00D	addi x11, x11, 1	Count up toward zero
00E	bnez x11, neg_loop	Repeat negative loop if counter is not zero
00F	addi x10, x10, 0	Done instruction

Table A2: Program instruction map used for the Lab 7 hardware demo.

A.3 Code and Testing Plan

The PC branch logic should reset PC to 0. On each BTN1 clock press, the PC should increment by 1 when take_branch is 0. When take_branch is 1, the PC should update to PC + signed offset. The positive multiplier test should produce x10 = 000C for 4 times 3.

Test	Expected result
Reset	PC becomes 000
Normal instruction	PC increments by 1
Positive loop branch	PC 008 branches back to PC 006 while x11 is not 0
Loop finished	PC 008 does not branch when x11 becomes 0
Done branch	PC 009 branches to PC 00F
Final product	x10 reaches 000C during the final positive-loop add

Table A3: Expected hardware test behavior for the Lab 7 demo program.

Test	Multiplicand x8	Multiplier x9	Expected product x10	Path
1	4	3	12 / 000C	Positive loop
2	4	-3	-12 / FFF4	Negative setup and loop
3	7	0	0 / 0000	Zero branch

Table A4: Signed multiplier test cases considered for the prelab.

Appendix B: Design Program Files

B.1 Program Counter

```
module program_counter(
   input  logic clk,
   input  logic rst,
   input  logic take_branch,
   input  logic signed [8:0] offset,
   output logic [8:0] pc
);
always_ff @(posedge clk or posedge rst) begin
   if (rst) begin
      pc <= 9'd0;
   end
   else begin
      if (take_branch) begin
         pc <= pc + offset;
      end
      else begin
         pc <= pc + 9'd1;
      end
   end
end
endmodule
```

B.2 Instruction Decoder Sign Extension and Branch Notes

The decoder sign-extends nzimm for addi and srai and sign-extends the 9-bit offset for branch instructions. The branch instructions set take_branch high only when the branch condition is true.

```
// addi: sign extend nzimm
instr_i = {{10{nzimm[5]}}, nzimm};
```

```
// srai / slli: use nzimm as the shift amount
instr_i = {{10{nzimm[5]}}, nzimm};
// beqz / bnez: sign extend the branch offset
instr_i = {{7{offset[8]}}, offset};
```

B.3 Assembly Program

```
init:
   addi x8, x8, 4         # set x8 to 4
   addi x9, x9, 3         # set x9 to 3
   add  x11, x11, x9      # copy x9 into counter x11
   beqz x11, done         # if counter is 0, go to done
   srai x9, x9, 7         # check if multiplier is negative
   bnez x9, neg_setup     # if negative, go to negative setup
pos_loop:
   add  x10, x10, x8      # add x8 to product x10
   addi x11, x11, -1      # subtract 1 from counter
   bnez x11, pos_loop     # repeat if counter is not 0
   beqz x11, done         # go to done when counter is 0
neg_setup:
   xor  x8, x8, x9        # flip x8 bits
   addi x8, x8, 1         # make x8 negative
neg_loop:
   add  x10, x10, x8      # add negative x8 to product
   addi x11, x11, 1       # add 1 to counter
   bnez x11, neg_loop     # repeat if counter is not 0
done:
   addi x10, x10, 0       # keep final result in x10
```

B.4 output.coe

```
memory_initialization_radix=16;
memory_initialization_vector=
0411,
048d,
95a6,
cd81,
849d,
e489,
9522,
15fd,
fdf5,
c591,
8c25,
0405,
9522,
0585,
fdf5,
852a;
```

Appendix C: Output Screen Captures

All hardware evidence below was captured from the Vivado Hardware Manager VIO. BTN1 was used as the step clock. BTN0 was used as reset. The displayed PC, instruction, opcode, take_branch, ALU output, and register write data signals show the program executing in hardware.

Figure	PC	Key evidence
C1	000	Instruction 0411 loaded; x8 set to 4
C2	001	Instruction 048D loaded; x9 set to 3
C3	002	Instruction 95A6 copies multiplier into x11

C4	003	beqz checked; branch not taken because x11 is nonzero
C5	004	srai sign check for positive multiplier
C6	005	bnez not taken; negative path skipped
C7	006	First add creates product 0004
C8	007	Counter decrements
C9	008	bnez branches back while counter is nonzero
C10	006	Second add creates product 0008
C11	007	Counter decrements again
C12	008	bnez loop control continues
C13	008	bnez stops branching once counter reaches 0
C14	009	beqz branches to done
C15	00F	done address reached after loop
C16	010	program moves past final stored instruction

Table C1: Summary of VIO hardware screenshots.

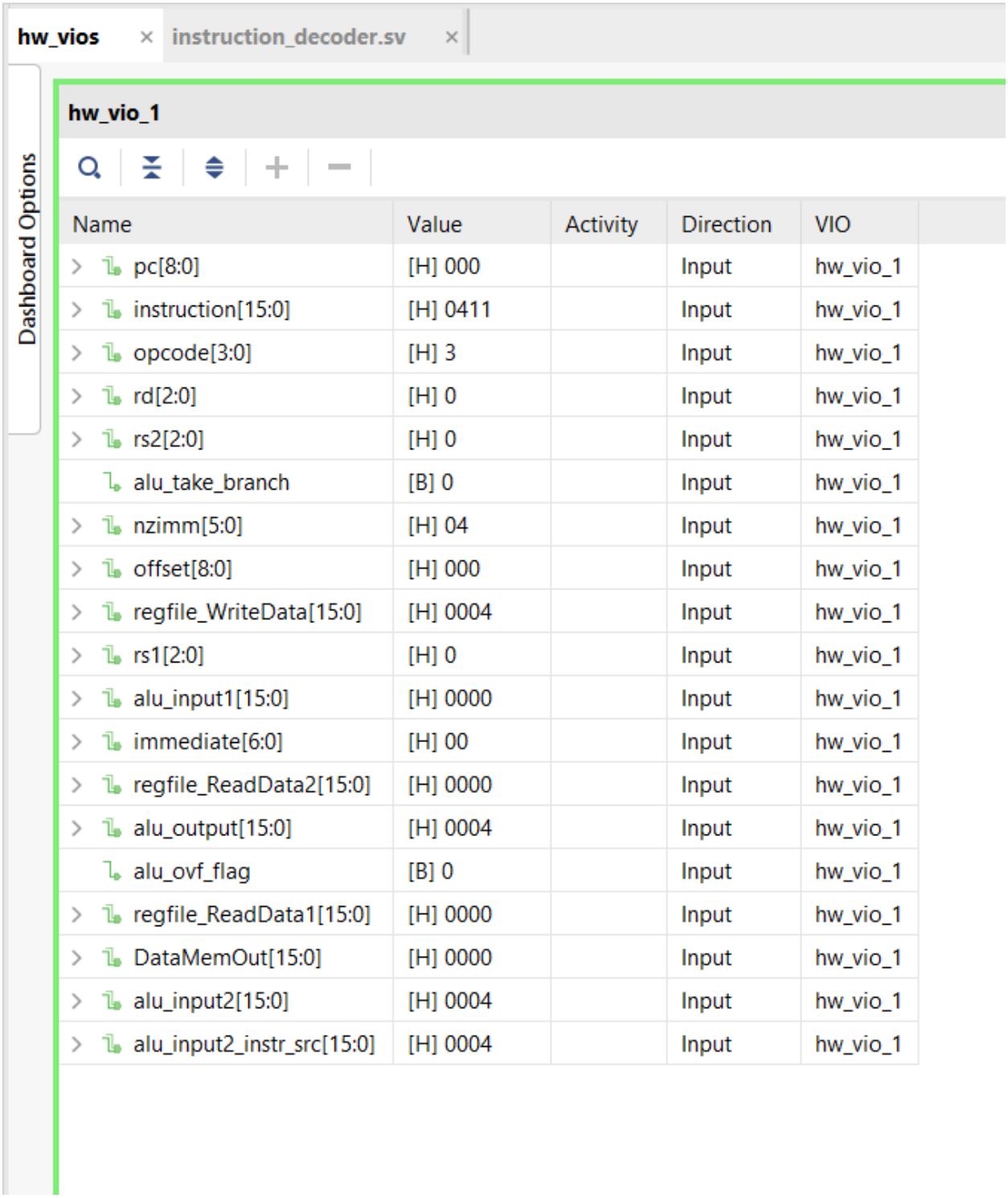
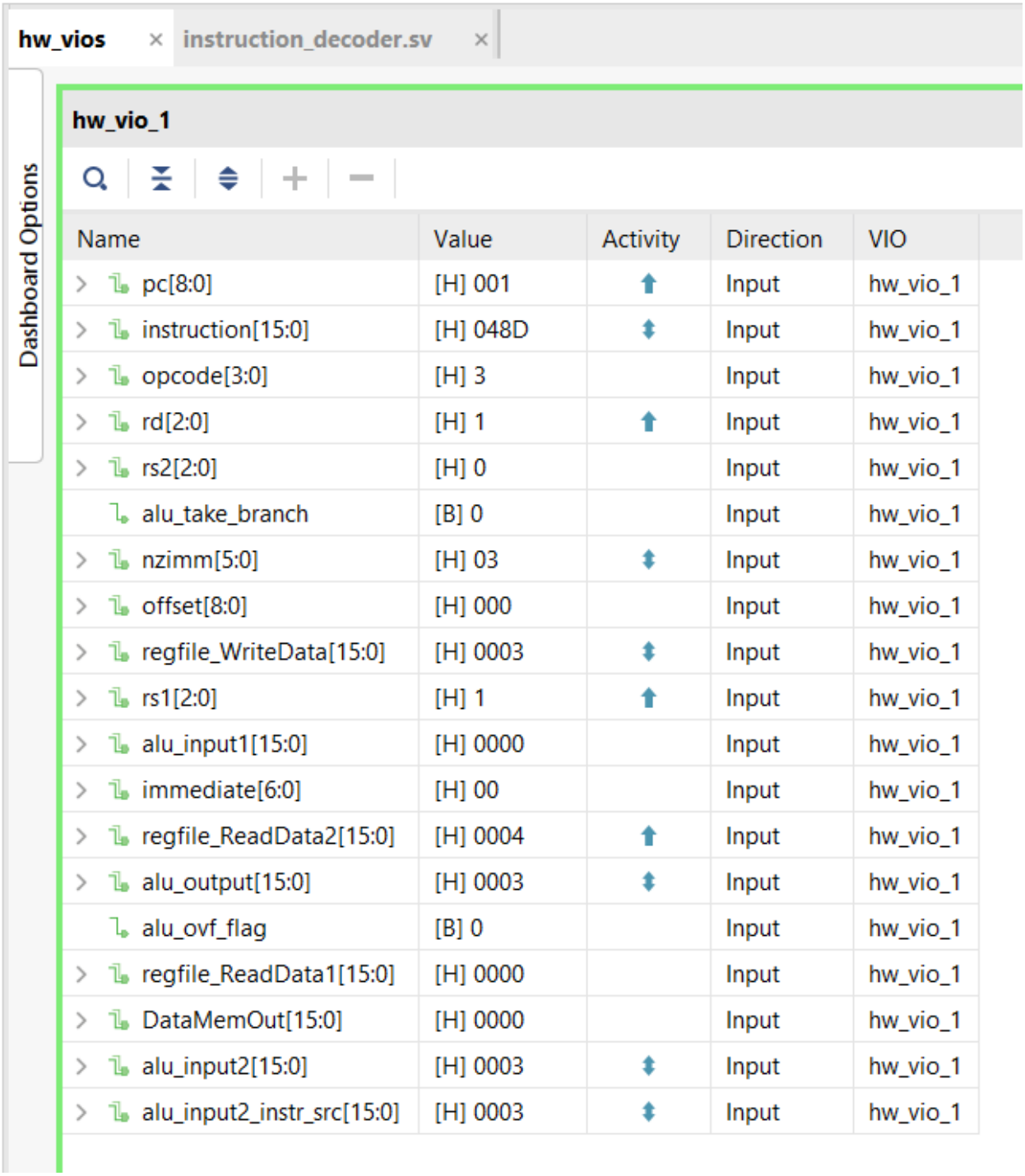


Figure C1: PC 000, instruction 16'h0411. The first instruction sets x8 to 4, which is the multiplicand.



Name	Value	Activity	Direction	VIO
pc[8:0]	[H] 001		Input	hw_vio_1
instruction[15:0]	[H] 048D		Input	hw_vio_1
opcode[3:0]	[H] 3		Input	hw_vio_1
rd[2:0]	[H] 1		Input	hw_vio_1
rs2[2:0]	[H] 0		Input	hw_vio_1
alu_take_branch	[B] 0		Input	hw_vio_1
nzimm[5:0]	[H] 03		Input	hw_vio_1
offset[8:0]	[H] 000		Input	hw_vio_1
regfile_WriteData[15:0]	[H] 0003		Input	hw_vio_1
rs1[2:0]	[H] 1		Input	hw_vio_1
alu_input1[15:0]	[H] 0000		Input	hw_vio_1
immediate[6:0]	[H] 00		Input	hw_vio_1
regfile_ReadData2[15:0]	[H] 0004		Input	hw_vio_1
alu_output[15:0]	[H] 0003		Input	hw_vio_1
alu_ovf_flag	[B] 0		Input	hw_vio_1
regfile_ReadData1[15:0]	[H] 0000		Input	hw_vio_1
DataMemOut[15:0]	[H] 0000		Input	hw_vio_1
alu_input2[15:0]	[H] 0003		Input	hw_vio_1
alu_input2_instr_src[15:0]	[H] 0003		Input	hw_vio_1

Figure C2: PC 001, instruction 16'h048D. The second instruction sets x9 to 3, which is the multiplier.

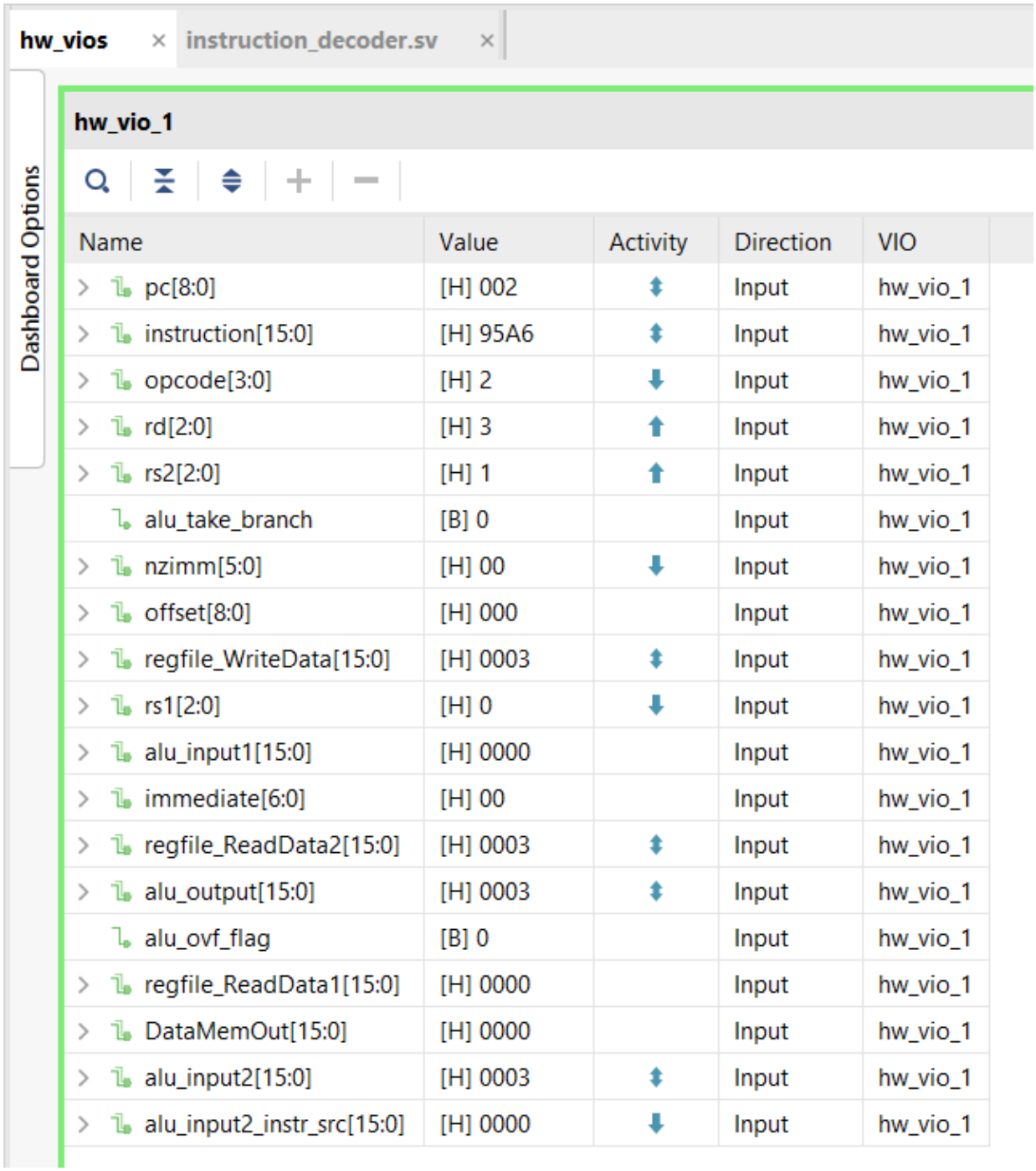
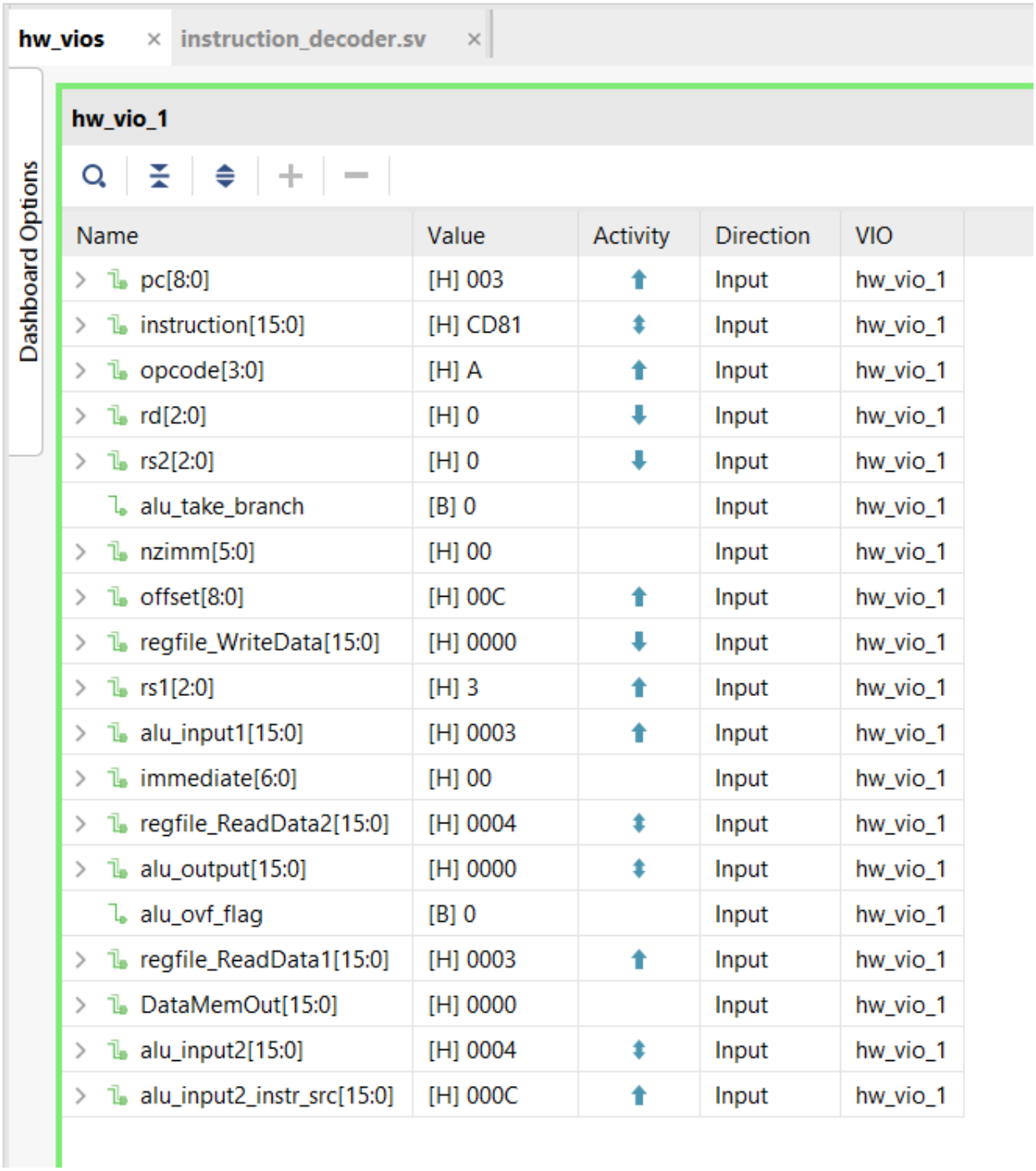


Figure C3: PC 002, instruction 16'h95A6. The multiplier is copied into x11, so the loop counter becomes 3.



Name	Value	Activity	Direction	VIO
pc[8:0]	[H] 003		Input	hw_vio_1
instruction[15:0]	[H] CD81		Input	hw_vio_1
opcode[3:0]	[H] A		Input	hw_vio_1
rd[2:0]	[H] 0		Input	hw_vio_1
rs2[2:0]	[H] 0		Input	hw_vio_1
alu_take_branch	[B] 0		Input	hw_vio_1
nzimm[5:0]	[H] 00		Input	hw_vio_1
offset[8:0]	[H] 00C		Input	hw_vio_1
regfile_WriteData[15:0]	[H] 0000		Input	hw_vio_1
rs1[2:0]	[H] 3		Input	hw_vio_1
alu_input1[15:0]	[H] 0003		Input	hw_vio_1
immediate[6:0]	[H] 00		Input	hw_vio_1
regfile_ReadData2[15:0]	[H] 0004		Input	hw_vio_1
alu_output[15:0]	[H] 0000		Input	hw_vio_1
alu_ovf_flag	[B] 0		Input	hw_vio_1
regfile_ReadData1[15:0]	[H] 0003		Input	hw_vio_1
DataMemOut[15:0]	[H] 0000		Input	hw_vio_1
alu_input2[15:0]	[H] 0004		Input	hw_vio_1
alu_input2_instr_src[15:0]	[H] 000C		Input	hw_vio_1

Figure C4: PC 003, instruction 16'hCD81. The beqz check is not taken because x11 is not zero.

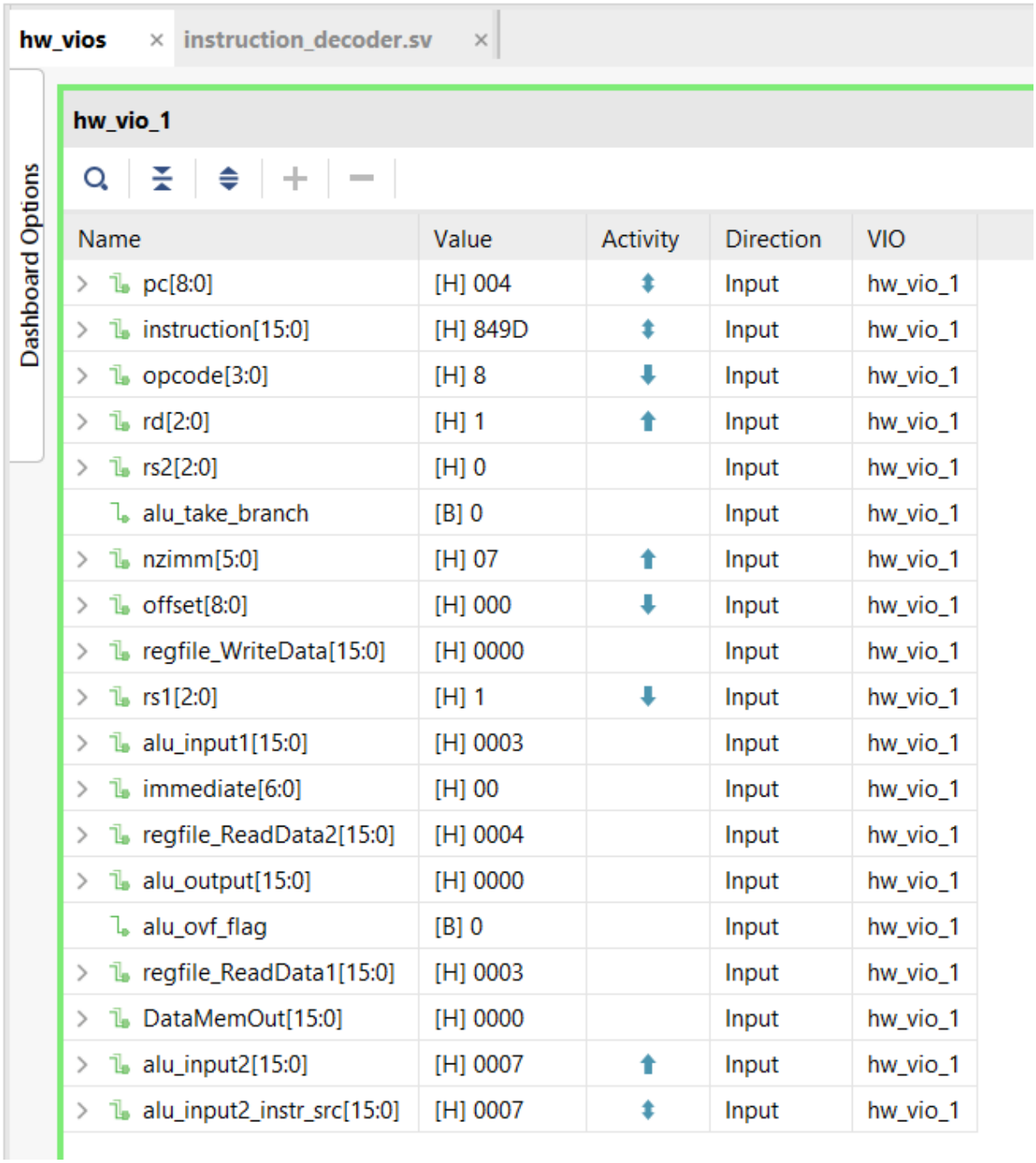


Figure C5: PC 004, instruction 16'h849D. The srai instruction checks the sign of the multiplier.

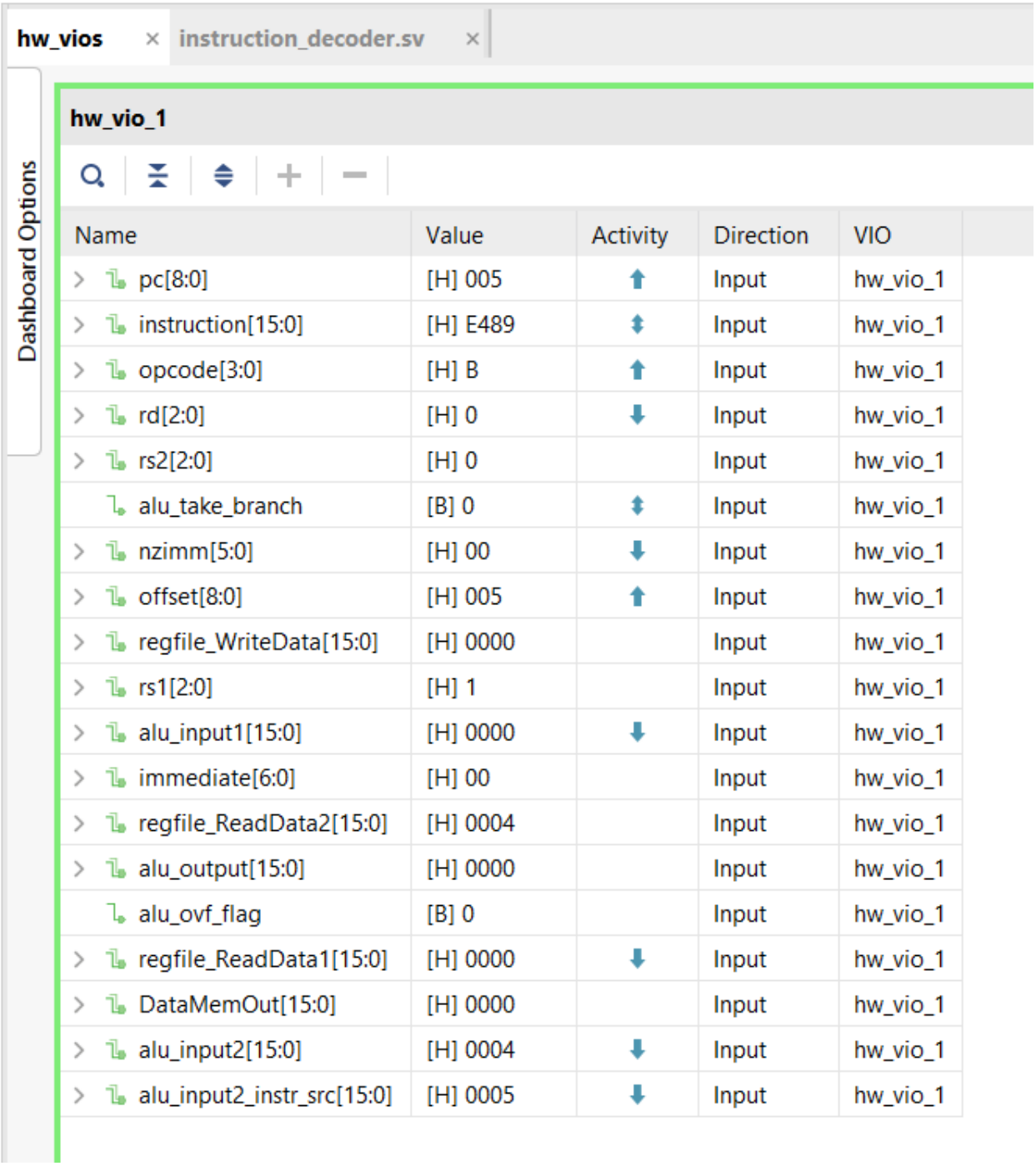
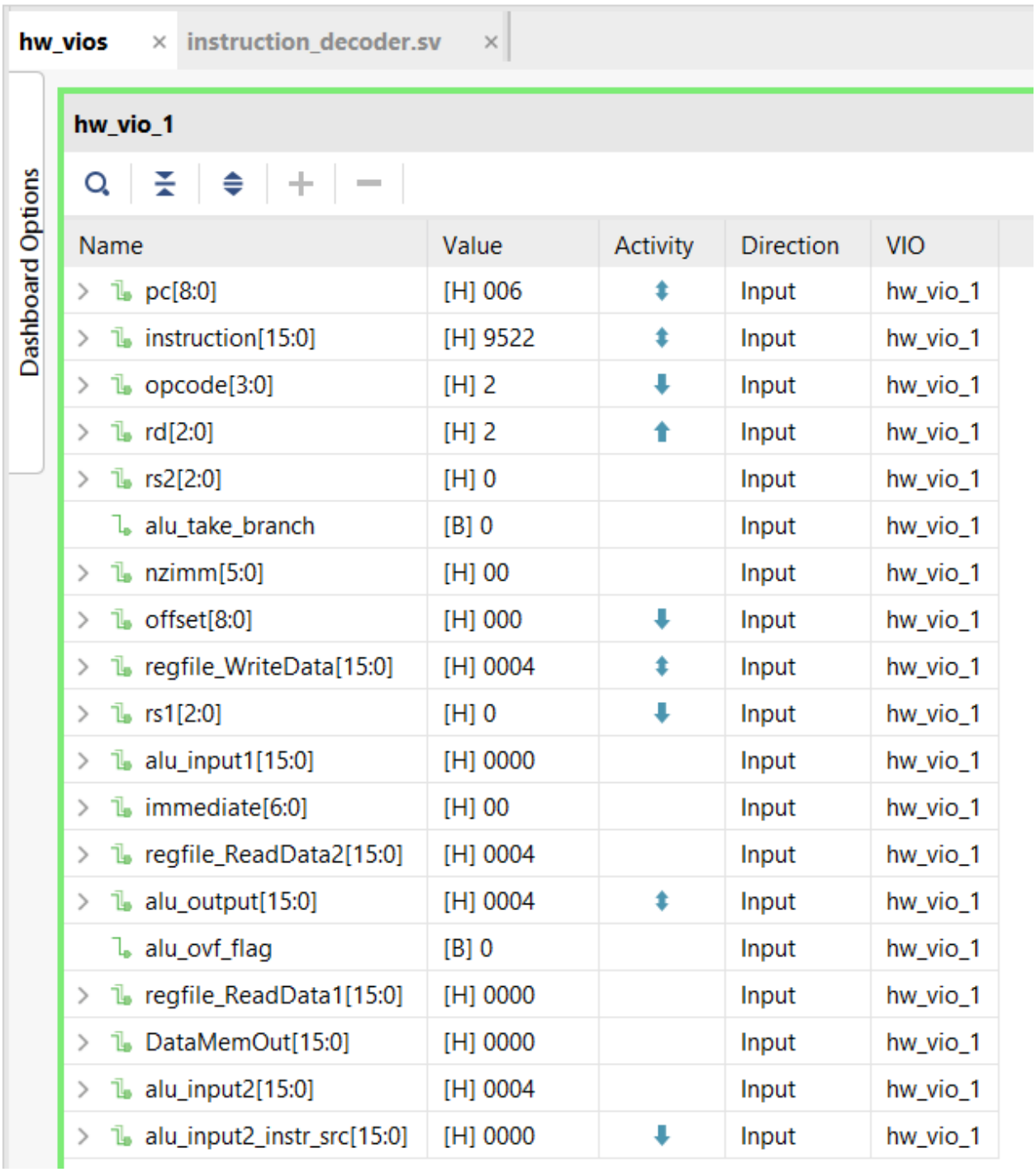
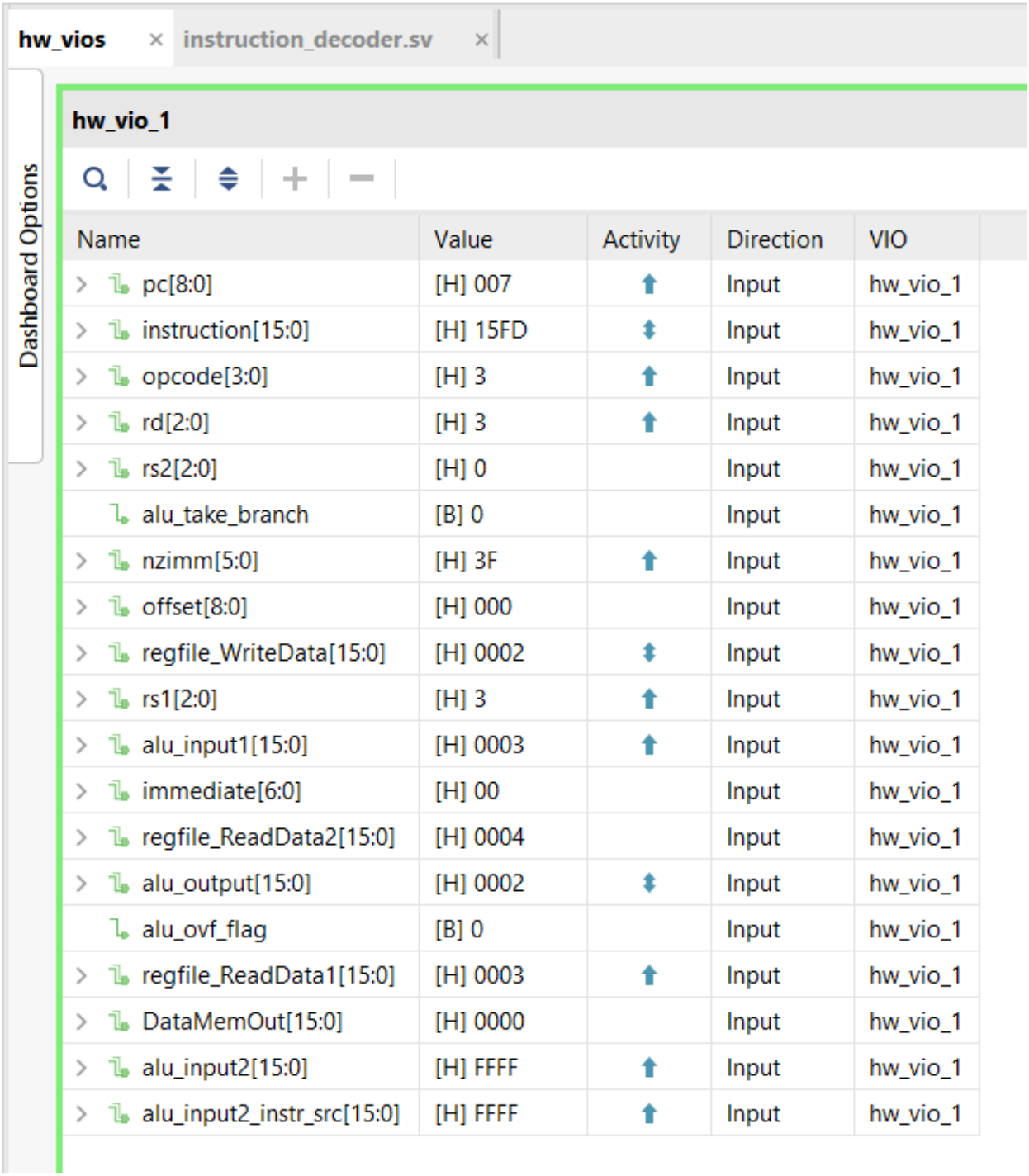


Figure C6: PC 005, instruction 16'hE489. The bnez negative-branch check is not taken because the multiplier is positive.



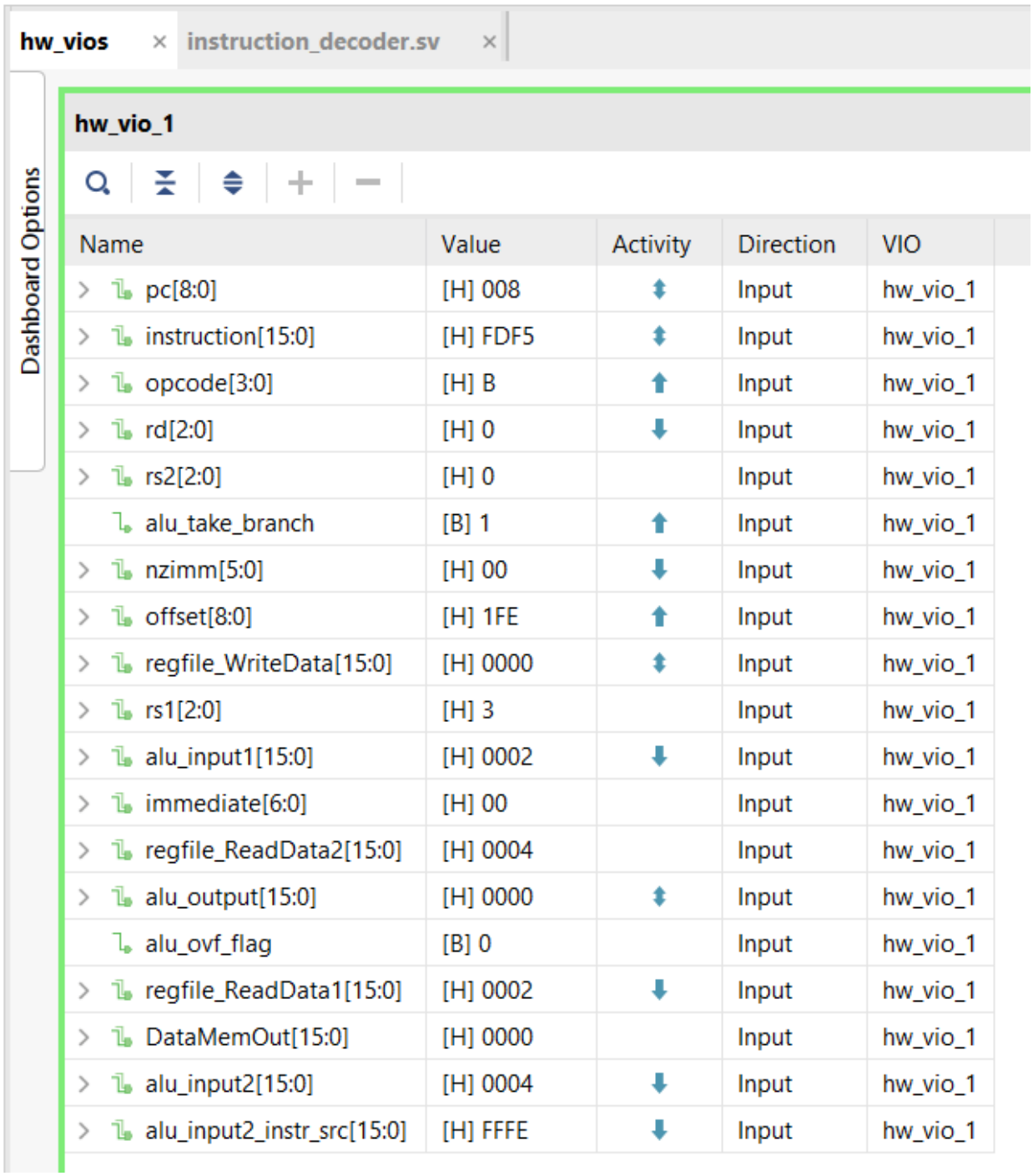
Name	Value	Activity	Direction	VIO
pc[8:0]	[H] 006	↕	Input	hw_vio_1
instruction[15:0]	[H] 9522	↕	Input	hw_vio_1
opcode[3:0]	[H] 2	↓	Input	hw_vio_1
rd[2:0]	[H] 2	↑	Input	hw_vio_1
rs2[2:0]	[H] 0		Input	hw_vio_1
alu_take_branch	[B] 0		Input	hw_vio_1
nzimm[5:0]	[H] 00		Input	hw_vio_1
offset[8:0]	[H] 000	↓	Input	hw_vio_1
regfile_WriteData[15:0]	[H] 0004	↕	Input	hw_vio_1
rs1[2:0]	[H] 0	↓	Input	hw_vio_1
alu_input1[15:0]	[H] 0000		Input	hw_vio_1
immediate[6:0]	[H] 00		Input	hw_vio_1
regfile_ReadData2[15:0]	[H] 0004		Input	hw_vio_1
alu_output[15:0]	[H] 0004	↕	Input	hw_vio_1
alu_ovf_flag	[B] 0		Input	hw_vio_1
regfile_ReadData1[15:0]	[H] 0000		Input	hw_vio_1
DataMemOut[15:0]	[H] 0000		Input	hw_vio_1
alu_input2[15:0]	[H] 0004		Input	hw_vio_1
alu_input2_instr_src[15:0]	[H] 0000	↓	Input	hw_vio_1

Figure C7: PC 006, instruction 16'h9522. The first positive-loop add produces x10 = 0004.



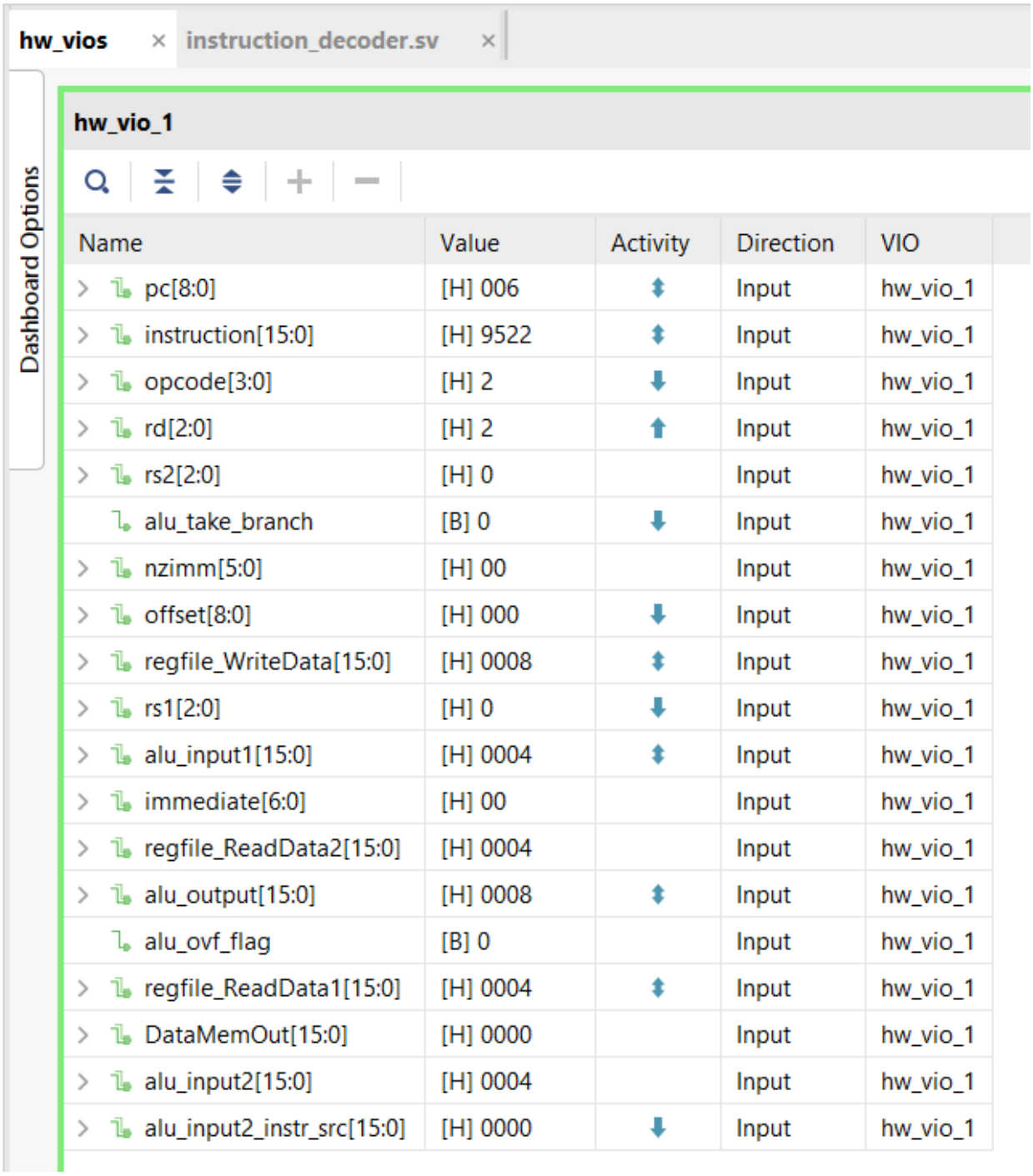
Name	Value	Activity	Direction	VIO
pc[8:0]	[H] 007	↑	Input	hw_vio_1
instruction[15:0]	[H] 15FD	↕	Input	hw_vio_1
opcode[3:0]	[H] 3	↑	Input	hw_vio_1
rd[2:0]	[H] 3	↑	Input	hw_vio_1
rs2[2:0]	[H] 0		Input	hw_vio_1
alu_take_branch	[B] 0		Input	hw_vio_1
nzimm[5:0]	[H] 3F	↑	Input	hw_vio_1
offset[8:0]	[H] 000		Input	hw_vio_1
regfile_WriteData[15:0]	[H] 0002	↕	Input	hw_vio_1
rs1[2:0]	[H] 3	↑	Input	hw_vio_1
alu_input1[15:0]	[H] 0003	↑	Input	hw_vio_1
immediate[6:0]	[H] 00		Input	hw_vio_1
regfile_ReadData2[15:0]	[H] 0004		Input	hw_vio_1
alu_output[15:0]	[H] 0002	↕	Input	hw_vio_1
alu_ovf_flag	[B] 0		Input	hw_vio_1
regfile_ReadData1[15:0]	[H] 0003	↑	Input	hw_vio_1
DataMemOut[15:0]	[H] 0000		Input	hw_vio_1
alu_input2[15:0]	[H] FFFF	↑	Input	hw_vio_1
alu_input2_instr_src[15:0]	[H] FFFF	↑	Input	hw_vio_1

Figure C8: PC 007, instruction 16'h15FD. The counter x11 is decremented using the -1 immediate.



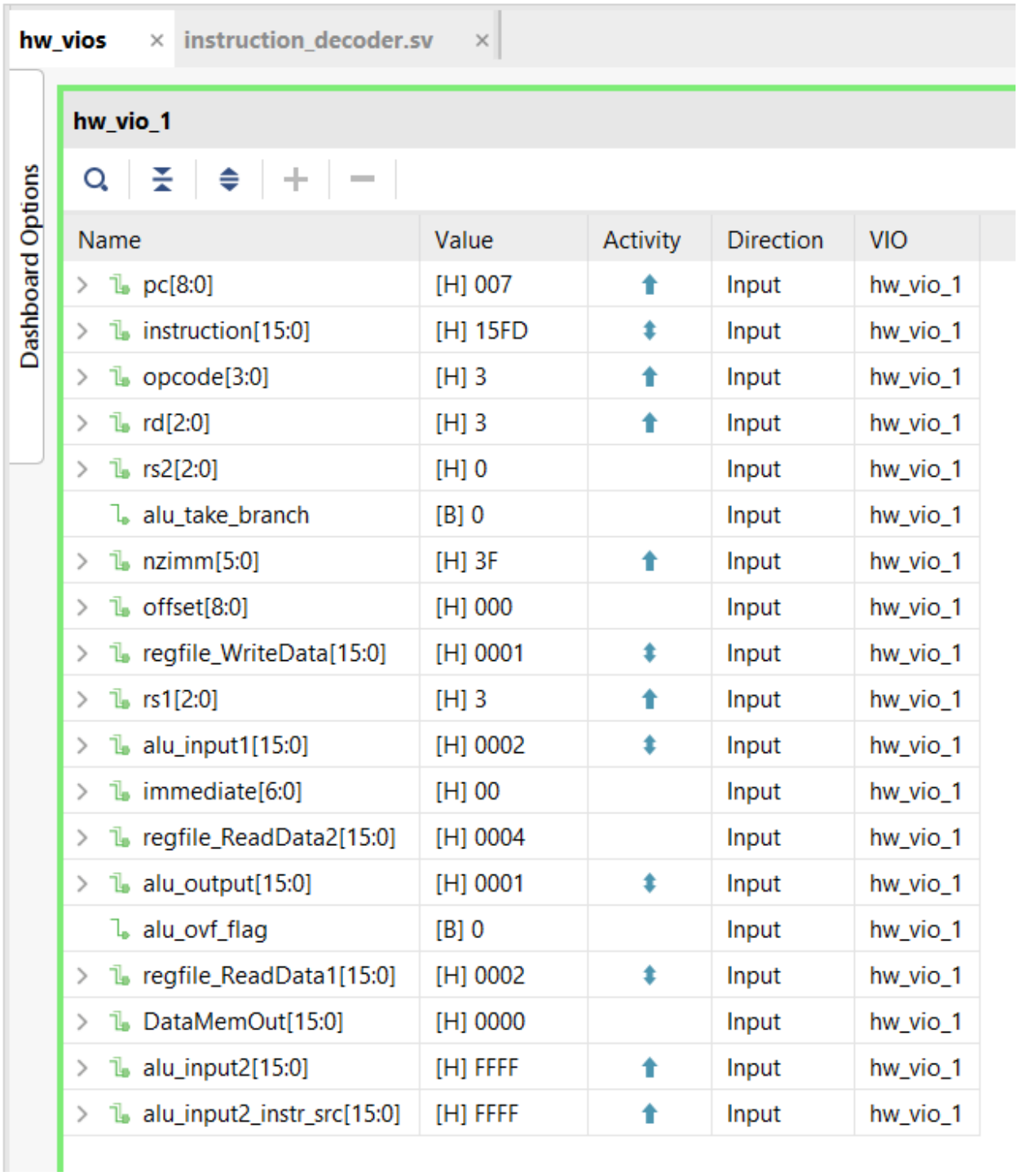
Name	Value	Activity	Direction	VIO
pc[8:0]	[H] 008	↕	Input	hw_vio_1
instruction[15:0]	[H] FDF5	↕	Input	hw_vio_1
opcode[3:0]	[H] B	↑	Input	hw_vio_1
rd[2:0]	[H] 0	↓	Input	hw_vio_1
rs2[2:0]	[H] 0		Input	hw_vio_1
alu_take_branch	[B] 1	↑	Input	hw_vio_1
nzimm[5:0]	[H] 00	↓	Input	hw_vio_1
offset[8:0]	[H] 1FE	↑	Input	hw_vio_1
regfile_WriteData[15:0]	[H] 0000	↕	Input	hw_vio_1
rs1[2:0]	[H] 3		Input	hw_vio_1
alu_input1[15:0]	[H] 0002	↓	Input	hw_vio_1
immediate[6:0]	[H] 00		Input	hw_vio_1
regfile_ReadData2[15:0]	[H] 0004		Input	hw_vio_1
alu_output[15:0]	[H] 0000	↕	Input	hw_vio_1
alu_ovf_flag	[B] 0		Input	hw_vio_1
regfile_ReadData1[15:0]	[H] 0002	↓	Input	hw_vio_1
DataMemOut[15:0]	[H] 0000		Input	hw_vio_1
alu_input2[15:0]	[H] 0004	↓	Input	hw_vio_1
alu_input2_instr_src[15:0]	[H] FFFE	↓	Input	hw_vio_1

Figure C9: PC 008, instruction 16'hFDF5. The bnez instruction asserts take_branch and returns to the loop while x11 is nonzero.



Name	Value	Activity	Direction	VIO
pc[8:0]	[H] 006		Input	hw_vio_1
instruction[15:0]	[H] 9522		Input	hw_vio_1
opcode[3:0]	[H] 2		Input	hw_vio_1
rd[2:0]	[H] 2		Input	hw_vio_1
rs2[2:0]	[H] 0		Input	hw_vio_1
alu_take_branch	[B] 0		Input	hw_vio_1
nzimm[5:0]	[H] 00		Input	hw_vio_1
offset[8:0]	[H] 000		Input	hw_vio_1
regfile_WriteData[15:0]	[H] 0008		Input	hw_vio_1
rs1[2:0]	[H] 0		Input	hw_vio_1
alu_input1[15:0]	[H] 0004		Input	hw_vio_1
immediate[6:0]	[H] 00		Input	hw_vio_1
regfile_ReadData2[15:0]	[H] 0004		Input	hw_vio_1
alu_output[15:0]	[H] 0008		Input	hw_vio_1
alu_ovf_flag	[B] 0		Input	hw_vio_1
regfile_ReadData1[15:0]	[H] 0004		Input	hw_vio_1
DataMemOut[15:0]	[H] 0000		Input	hw_vio_1
alu_input2[15:0]	[H] 0004		Input	hw_vio_1
alu_input2_instr_src[15:0]	[H] 0000		Input	hw_vio_1

Figure C10: PC 006 again. The second positive-loop add produces x10 = 0008.



Name	Value	Activity	Direction	VIO
pc[8:0]	[H] 007		Input	hw_vio_1
instruction[15:0]	[H] 15FD		Input	hw_vio_1
opcode[3:0]	[H] 3		Input	hw_vio_1
rd[2:0]	[H] 3		Input	hw_vio_1
rs2[2:0]	[H] 0		Input	hw_vio_1
alu_take_branch	[B] 0		Input	hw_vio_1
nzimm[5:0]	[H] 3F		Input	hw_vio_1
offset[8:0]	[H] 000		Input	hw_vio_1
regfile_WriteData[15:0]	[H] 0001		Input	hw_vio_1
rs1[2:0]	[H] 3		Input	hw_vio_1
alu_input1[15:0]	[H] 0002		Input	hw_vio_1
immediate[6:0]	[H] 00		Input	hw_vio_1
regfile_ReadData2[15:0]	[H] 0004		Input	hw_vio_1
alu_output[15:0]	[H] 0001		Input	hw_vio_1
alu_ovf_flag	[B] 0		Input	hw_vio_1
regfile_ReadData1[15:0]	[H] 0002		Input	hw_vio_1
DataMemOut[15:0]	[H] 0000		Input	hw_vio_1
alu_input2[15:0]	[H] FFFF		Input	hw_vio_1
alu_input2_instr_src[15:0]	[H] FFFF		Input	hw_vio_1

Figure C11: PC 007 again. The counter is decremented toward zero.

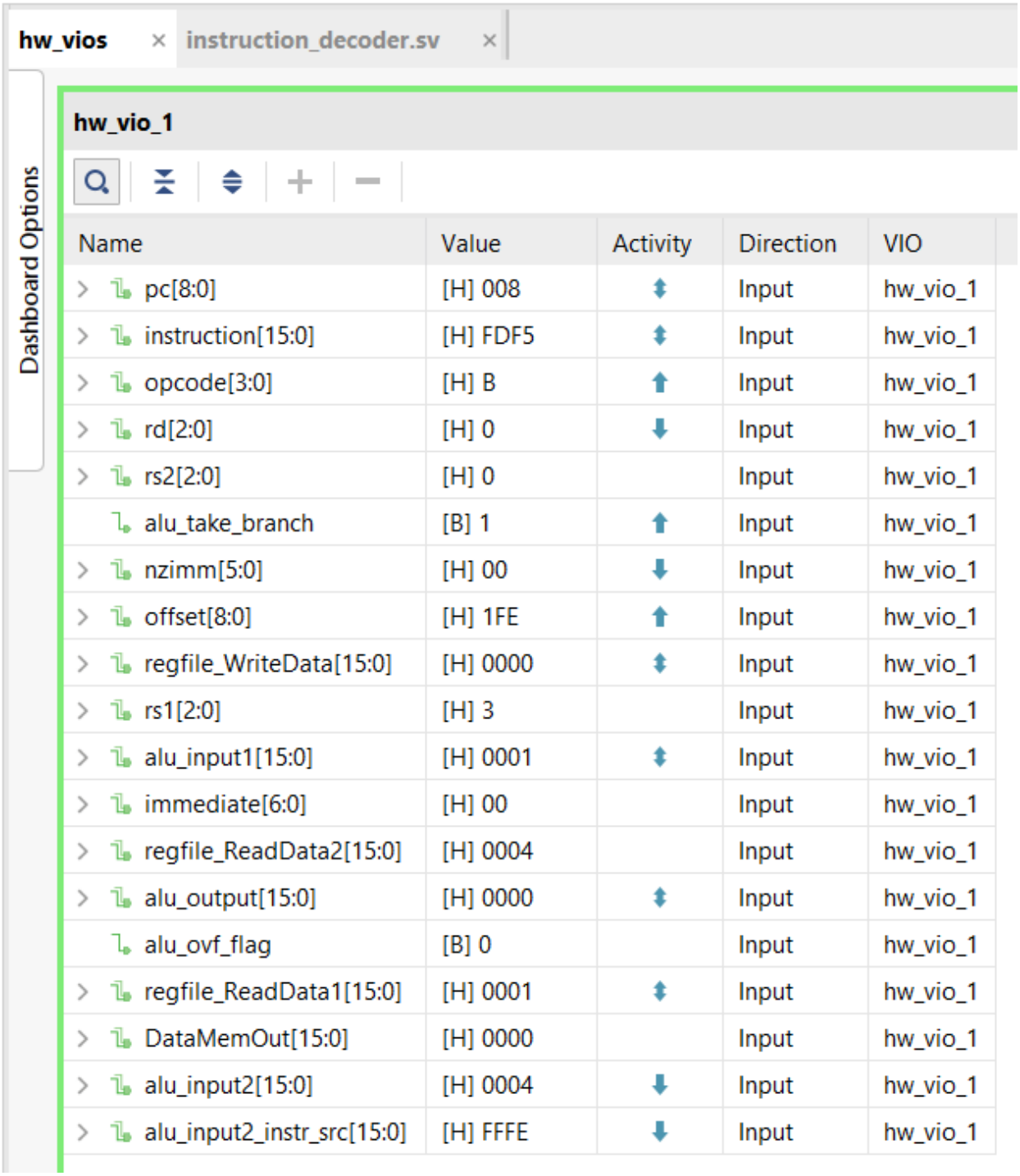


Figure C12: PC 008 again. The bnez loop control is evaluated after another counter update.

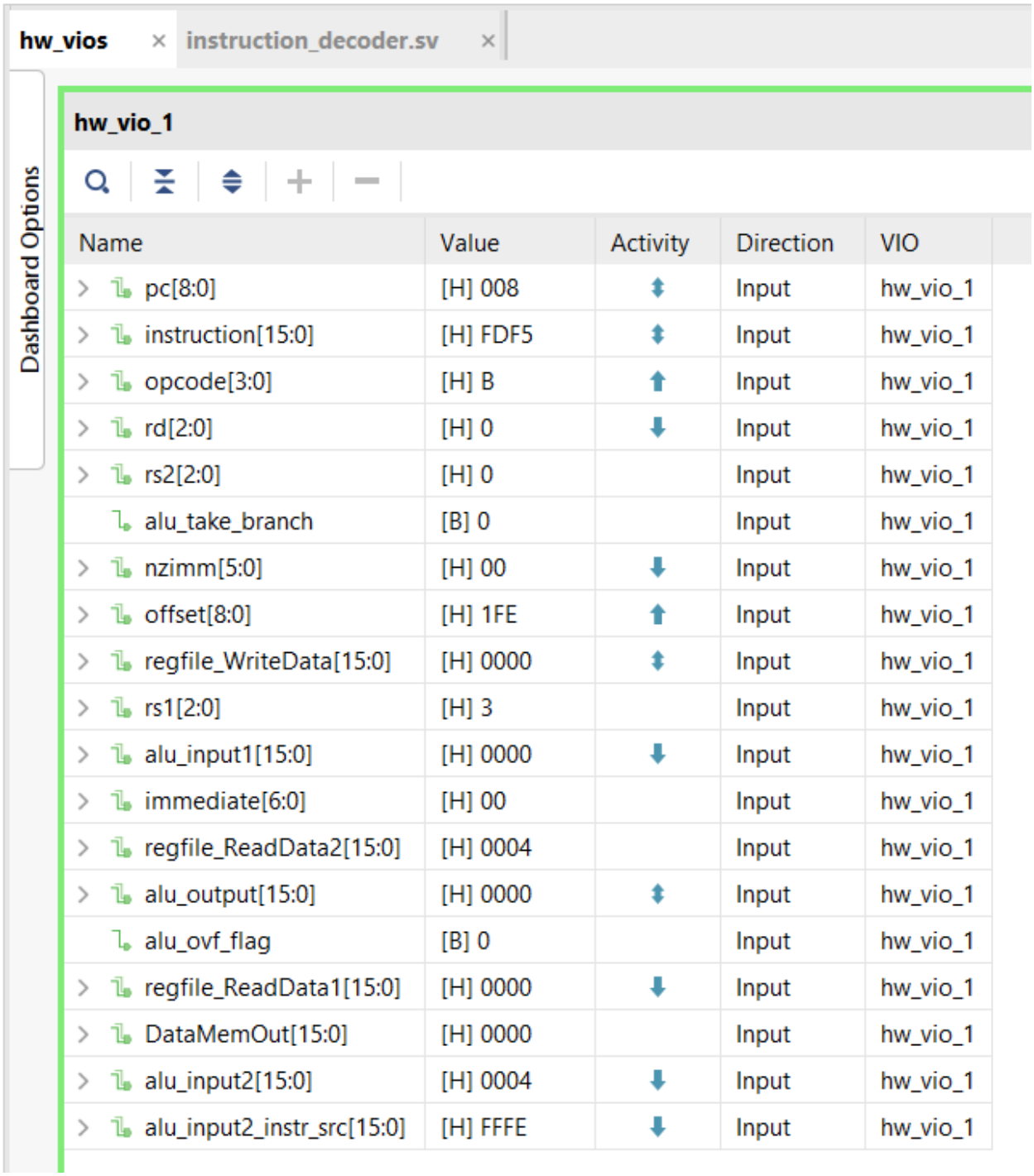
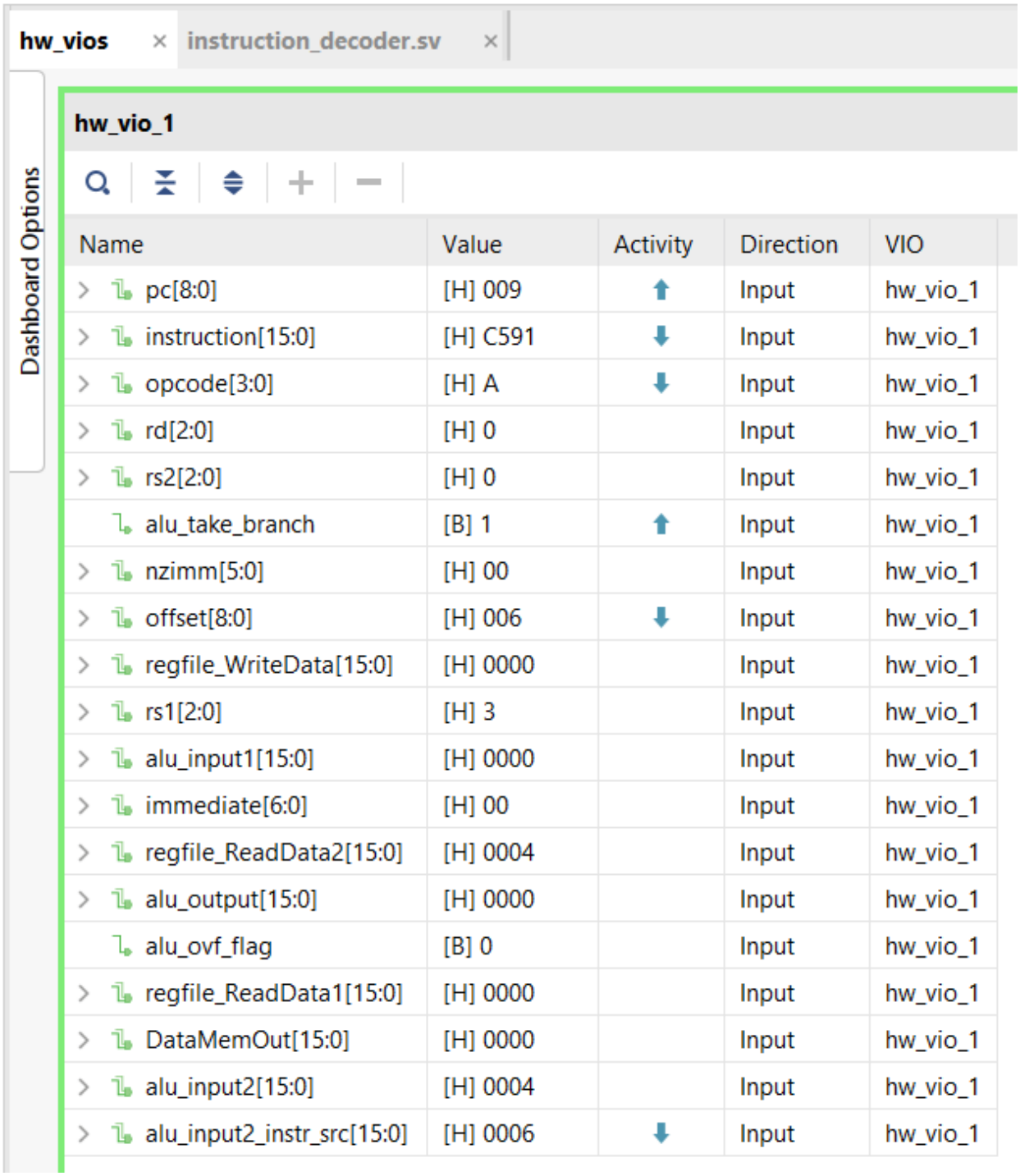
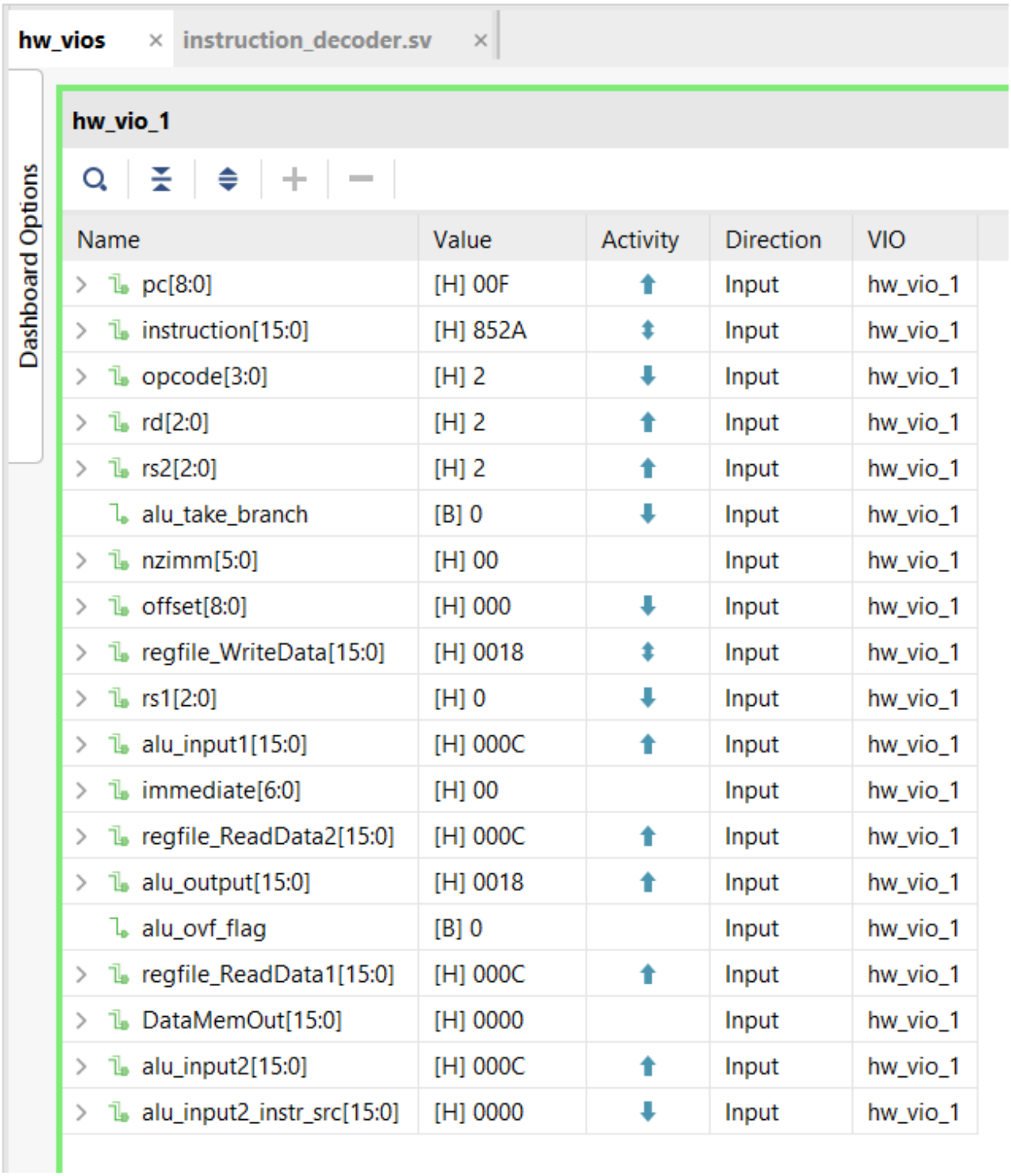


Figure C13: PC 008 at the final loop check. With the counter at zero, take_branch is not asserted.



Name	Value	Activity	Direction	VIO
pc[8:0]	[H] 009	↑	Input	hw_vio_1
instruction[15:0]	[H] C591	↓	Input	hw_vio_1
opcode[3:0]	[H] A	↓	Input	hw_vio_1
rd[2:0]	[H] 0		Input	hw_vio_1
rs2[2:0]	[H] 0		Input	hw_vio_1
alu_take_branch	[B] 1	↑	Input	hw_vio_1
nzimm[5:0]	[H] 00		Input	hw_vio_1
offset[8:0]	[H] 006	↓	Input	hw_vio_1
regfile_WriteData[15:0]	[H] 0000		Input	hw_vio_1
rs1[2:0]	[H] 3		Input	hw_vio_1
alu_input1[15:0]	[H] 0000		Input	hw_vio_1
immediate[6:0]	[H] 00		Input	hw_vio_1
regfile_ReadData2[15:0]	[H] 0004		Input	hw_vio_1
alu_output[15:0]	[H] 0000		Input	hw_vio_1
alu_ovf_flag	[B] 0		Input	hw_vio_1
regfile_ReadData1[15:0]	[H] 0000		Input	hw_vio_1
DataMemOut[15:0]	[H] 0000		Input	hw_vio_1
alu_input2[15:0]	[H] 0004		Input	hw_vio_1
alu_input2_instr_src[15:0]	[H] 0006	↓	Input	hw_vio_1

Figure C14: PC 009, instruction 16'hC591. The beqz instruction asserts take_branch and jumps to the done address.



Name	Value	Activity	Direction	VIO
pc[8:0]	[H] 00F	↑	Input	hw_vio_1
instruction[15:0]	[H] 852A	↕	Input	hw_vio_1
opcode[3:0]	[H] 2	↓	Input	hw_vio_1
rd[2:0]	[H] 2	↑	Input	hw_vio_1
rs2[2:0]	[H] 2	↑	Input	hw_vio_1
alu_take_branch	[B] 0	↓	Input	hw_vio_1
nzimm[5:0]	[H] 00		Input	hw_vio_1
offset[8:0]	[H] 000	↓	Input	hw_vio_1
regfile_WriteData[15:0]	[H] 0018	↕	Input	hw_vio_1
rs1[2:0]	[H] 0	↓	Input	hw_vio_1
alu_input1[15:0]	[H] 000C	↑	Input	hw_vio_1
immediate[6:0]	[H] 00		Input	hw_vio_1
regfile_ReadData2[15:0]	[H] 000C	↑	Input	hw_vio_1
alu_output[15:0]	[H] 0018	↑	Input	hw_vio_1
alu_ovf_flag	[B] 0		Input	hw_vio_1
regfile_ReadData1[15:0]	[H] 000C	↑	Input	hw_vio_1
DataMemOut[15:0]	[H] 0000		Input	hw_vio_1
alu_input2[15:0]	[H] 000C	↑	Input	hw_vio_1
alu_input2_instr_src[15:0]	[H] 0000	↓	Input	hw_vio_1

Figure C15: PC 00F, instruction 16'h852A. The done address is reached after the positive loop. The loop has produced x10 = 000C before this end step.

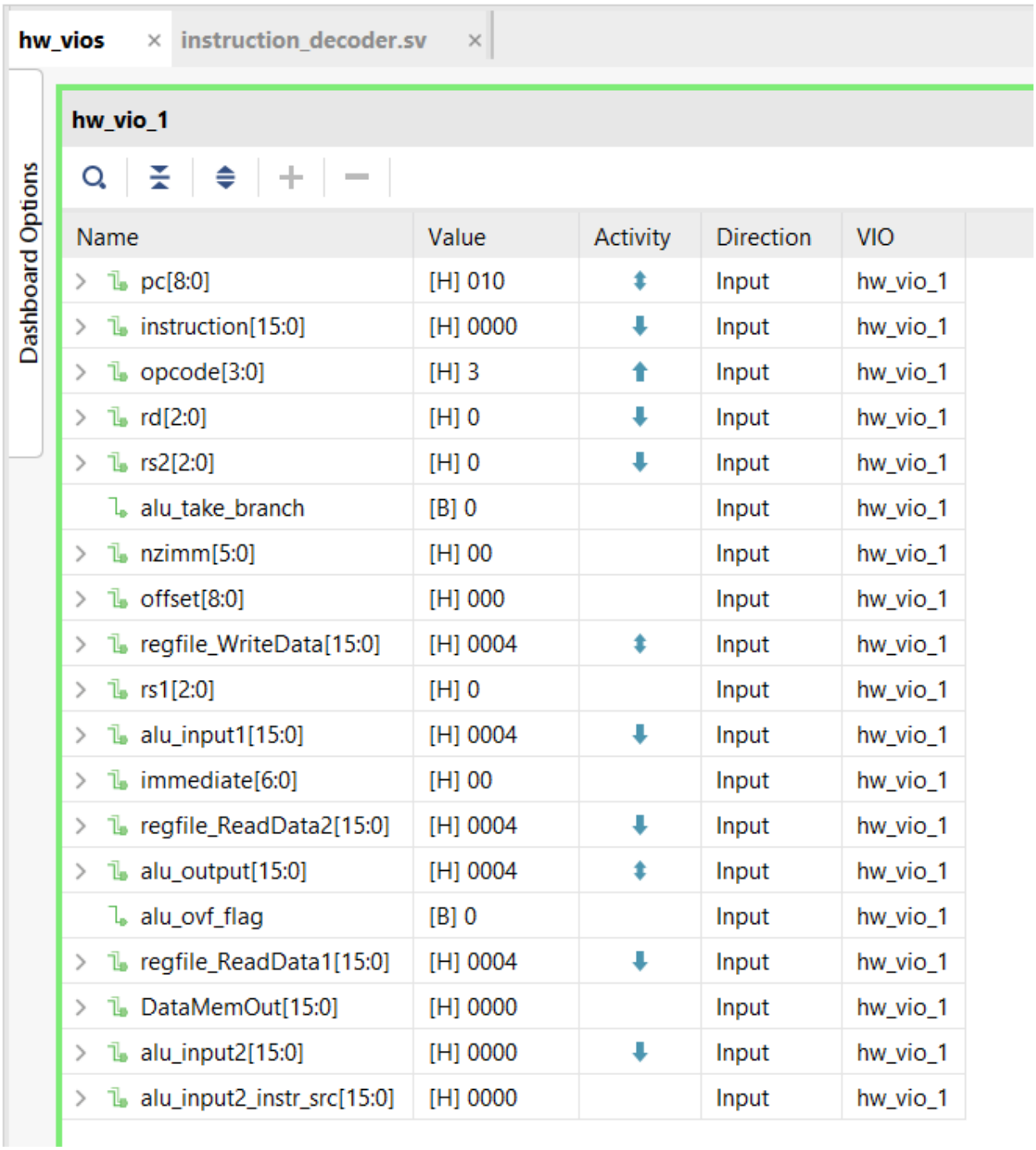


Figure C16: PC 010, instruction 16'h0000. The PC has moved past the loaded program instructions.